High-Performance Quantitative Pricing Engine

Разработчики ядра на C/C++, системные архитекторы

1 Описание системы

HP QPE это вычислительное ядро для высокочастотной оценки финансовых деривативов и расчета риск-метрик. Архитектура построена по **многоуровневому паттерну Layered Architecture**, что позволяет жестко изолировать платформо-зависимые оптимизации, например, SIMD-инструкции, управление иерархией кэш-памяти, от высокоуровневой финансовой бизнес-логики.

2 Структура системы

1. **Низкоуровневое ядро/HPC Core** базируется на кастомной реализации математических функций в виде библиотеки `MathLib`, векторизованной линейной алгебре и статистически валидированных генераторах псевдослучайных чисел.
2. **Слой моделирования/Stochastic Layer** предоставляет абстракции для стохастических дифференциальных уравнений (геометрическое броуновское движение) и вычислительные движки (симуляции Монте-Карло, биномиальные деревья).
3. **Уровень приложения/Application Layer** — финансовые калькуляторы: оценка Американских и Европейских опционов, API для расчета чувствительности портфеля / «греков».

3 Обоснование выбранных архитектурных решений

3.1 Отказ от стандартной `libm` в пользу кастомной `MathLib`

- *Причина:* Стандартные функции, например `std::log`, `std::exp`, вызывают сериализацию потока инструкций и ветвления, что блокирует автовекторизацию и создает вычислительный bottleneck при обходе миллионов узлов биномиального дерева.
- *Решение:* Реализован режим Enhanced Performance. Использован метод Table-Lookup с использованием 8-битной индексации мантиссы в связке с аппроксимацией минимаксными полиномами по схеме Эстрина. Это обеспечило 100% утилизацию FMA-портов процессора с математически гарантированной погрешностью < 3.5 ULP.

3.2 Иерархия памяти в линейной алгебре

- *Причина:* Перемножение матриц ковариации для корзинных опционов является строго Memory-Bound задачей. Наивный построчный доступ приводит к постоян-

ному вытеснению кэш-линий и деградации производительности.

- *Решение:* Внедрен Tiling матриц на блоки 64×64 для постоянного удержания рабочих данных в L1D-кэше. На уровне микро-кернелов применен Register Blocking (для процессоров, поддерживающих AVX2, утилизация 14 из 16 доступных YMM регистров) с использованием явных интринсиков AVX2 и AVX-512.

3.3 Паттерн распараллеливания RNG

- *Причина:* Использование потоков OpenMP в методе Монте-Карло с единым объектом RNG приводит к race condition данных. Использование инициализации потоков разными seed-значениями математически не гарантирует отсутствие пересечения последовательностей, то есть отсутствие независимости.

- *Решение:* Имплементирован алгоритм `discard` над ядром генератора MT19937. Это математически гарантирует независимость и непересекаемость потоков данных. Качество энтропии подтверждено встроенной двухуровневой системой тестирования (χ^2, Колмогоров-Смирнов, автокорреляция) со ссылками на уже существующие батареи тестов и литературу.

4 Стратегия тестирования и контроля качества

- Низкоуровневая библиотека `MathLib` покрывается тестами на всем диапазоне значений `float`. Валидация ULP-ошибок проводится относительно высокоточных эталонов. Для генерации минимаксных полиномов используются инструменты произвольной точности (например Sollya/MPFR).

- Для бизнес-логики (генерация биномиальных деревьев) применяется интервальное тестирование граничных состояний в зоне Deep In-The-Money с целью контроля и предотвращения эффекта катастрофической потери точности при вычислениях.

5 Правки

Осведомленность Application Layer об аппаратных оптимизациях

В чистой слоистой архитектуре **Application Layer** (она же бизнес-логика) действительно не должен содержать платформо-зависимого кода, например интринсиков. Однако для обеспечения высокой производительности он обязан быть спроектирован с учетом архитектуры железа (*Hardware-Aware API*), применяя парадигму Data-Oriented Design.

- **Взаимодействие с векторизацией / SIMD:** Application Layer не вызывает векторные инструкции напрямую, но он оперирует векторными интерфейсами. Вместо расчета одного опциона в цикле, он формирует выровненные массивы (*SoA* вместо *AoS*) и передает их вниз единым блоком. Слой *HPC Core* получает этот блок и применяет к нему SIMD-инструкции. Таким образом, бизнес-логика задает намерение на пакетную обработку, а ядро осуществляет её аппаратное выполнение.

- **Управление потоками / Multithreading:** Бизнес-логика управляет зернистостью задач. Она разбивает расчет портфеля из 10 000 опционов на логические блоки, например, по 500 штук и отправляет их в виде абстрактных тасок в *Hardware*

/ *OS Abstraction Layer*. Контроль над тем, к какому физическому ядру привязан поток, остается строго в зоне ответственности низкоуровневого thread pool.

Адаптация под мультиархитектурность (x86, ARM, AMX)

Если система должна поддерживать 2–3 архитектуры, включая матричные расширения типа Intel AMX, монолитный *HPC Core* должен быть преобразован в систему с **динамической диспетчеризацией**.

Изменения в архитектуре будут следующими:

- **Внедрение Hardware Abstraction Layer / Dispatcher:** Появляется компонент диспетчера. При старте, или на этапе компиляции через Function Multi-Versioning, система вызывает *CPU ID* и определяет доступные наборы инструкций.
- **Разделение на Backend-кернелы:** Математика и линейная алгебра реализуются через абстрактные интерфейсы (*ILinAlg*), за которыми скрываются специфичные реализации: *AVX2_Backend*, *NEON_Backend* (для ARM) и *AMX_Backend*.
- **Специфика поддержки Intel AMX:** Advanced Matrix Extensions кардинально меняет работу с памятью. В отличие от 1D-векторов AVX, AMX оперирует двумерными тайлами TMUL. Поэтому в слой диспетчеризации необходимо добавить компонент *Data Packer*. Его задача перед вызовом AMX-кернела переупаковывать стандартные Row-Major матрицы в специализированный блочный формат, оптимальный для загрузки в 2D-регистры AMX с учетом специфического шага stride, а также управлять конфигурацией тайлов, инструкция LDTILECFG.